HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO THỰC HÀNH
DỊCH NGƯỢC
AN TOÀN GAME
BENTOLOOP

Giảng viên hướng dẫn: Cao Minh Thắng
Lớp: D24CQGA02-B

Sinh viên **MSV**
Nguyễn Thành Vinh B24DCGA162

HÀ NỘI - 10/2026

MỤC LỤC

1 Mục tiêu và môi trường thực hành 2
2 Decompile APK và xác định thư viện 2
3 Tra cứu lớp LicenseVerifier trong dữ liệu dump 4
4 Import thư viện và gắn tên trong Ghidra 5
4.1 Nhận dạng thư viện ARM64 5
4.2 Quy đổi RVA sang địa chỉ Ghidra 6
4.3 Áp dụng tên từ script.json 7
5 Phân tích luồng kiểm tra license 8
5.1 Kiểm tra đầu vào và nhánh so sánh trực tiếp 8
5.2 Nhánh trả về thành công 8
5.3 Nhánh kiểm tra mảng mã hóa 9
6 Đối chiếu key và kiểm chứng trên APK gốc 10
6.1 Đọc nội dung StringLiteral_2770 10
6.2 Đối chiếu với stringliteral.json 10
6.3 Nhập key và vượt màn hình license 11
7 Trạng thái nhiệm vụ sửa và build APK 12
8 Nhận xét an toàn và phương án phòng tránh 12
8.1 Rủi ro của nhánh so sánh key trên client 12
8.2 Phương án giảm rủi ro 13
9 Kết luận 13

1 Mục tiêu và môi trường thực hành

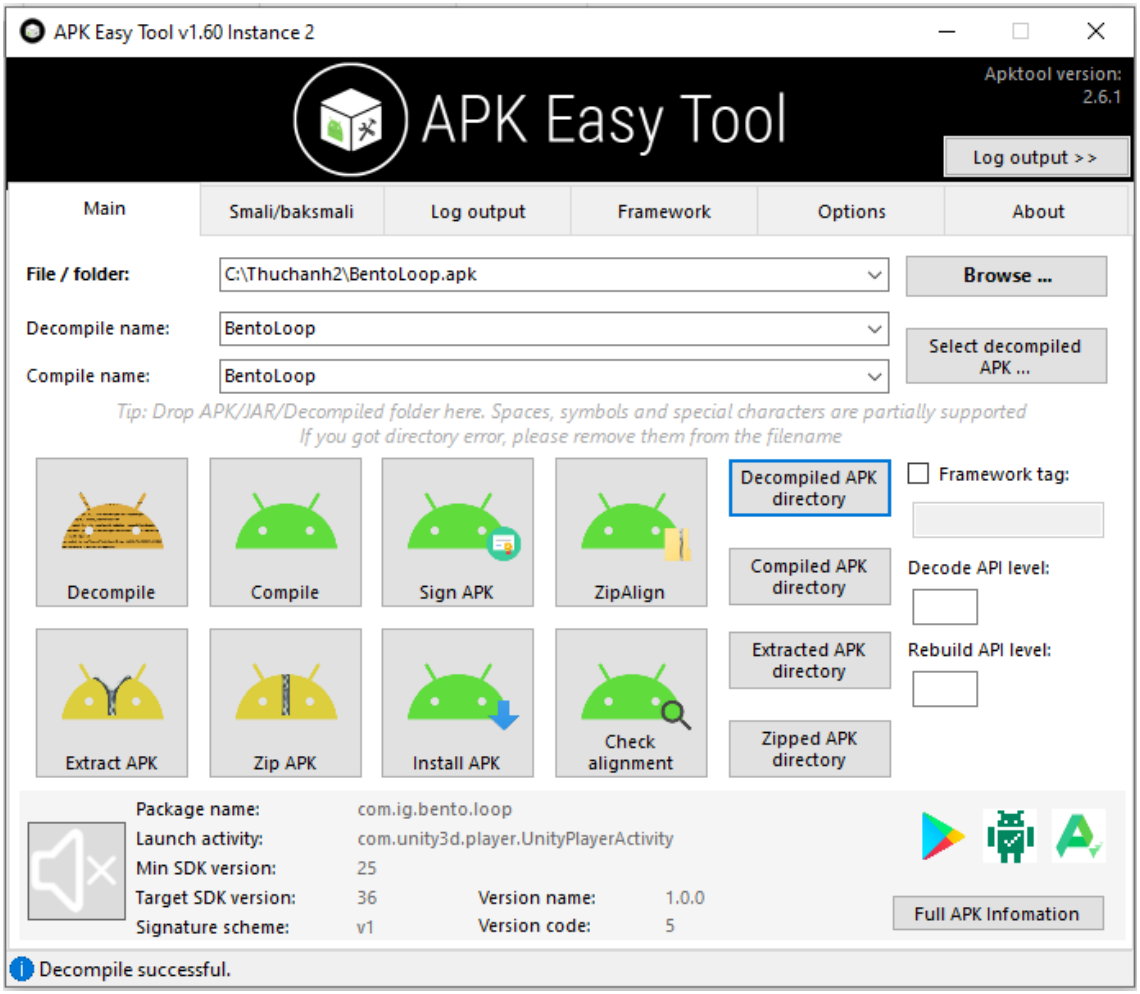
Bài thực hành phân tích cơ chế kiểm tra license của BentoLoop trên Android. Từ dữ liệu dump và mã native, đã tìm được chuỗi BENTO-EASY-2025 ở nhánh so sánh trực tiếp của VerifyLicense. Chuỗi này đã được nhập vào APK gốc để vượt màn hình license và vào Level 1. Phần sửa thư viện và build lại APK chưa được thực hiện.

Theo yêu cầu thực hành, mỗi bước cần nêu dữ liệu đầu vào, thao tác và kết quả kèm ảnh. Hai kết quả cần đạt là tìm key để vượt màn hình kiểm tra và sửa hoặc vòng tránh cơ chế kiểm tra rồi build lại APK. Báo cáo trình bày phần tìm key đã hoàn thành và phân biệt với nhiệm vụ sửa APK còn lại.

Thành phần	Thông tin sử dụng
Môi trường phân tích	Windows trong máy ảo VMware Workstation, tên thuchanhdichnguoc1
APK	BentoLoop.apk; package com.ig.bento.loop; phiên bản 1.0.0, version code 5
APK Easy Tool	Phiên bản 1.60, Apktool 2.6.1; dùng để Decompile APK
Dữ liệu IL2CPP	Gói libil2cpp_il2cppdump_20261007_0259 do giảng viên cung cấp
dnSpy và dump.cs	Tra cứu cấu trúc lớp, chữ ký hàm và địa chỉ RVA
Ghidra	Phiên bản 12.1.2; project Thuchanh2; thư viện ARM64
Kiểm chứng	Chạy Bento Loop trên Android, nhập key tìm được và kiểm tra vào Level 1

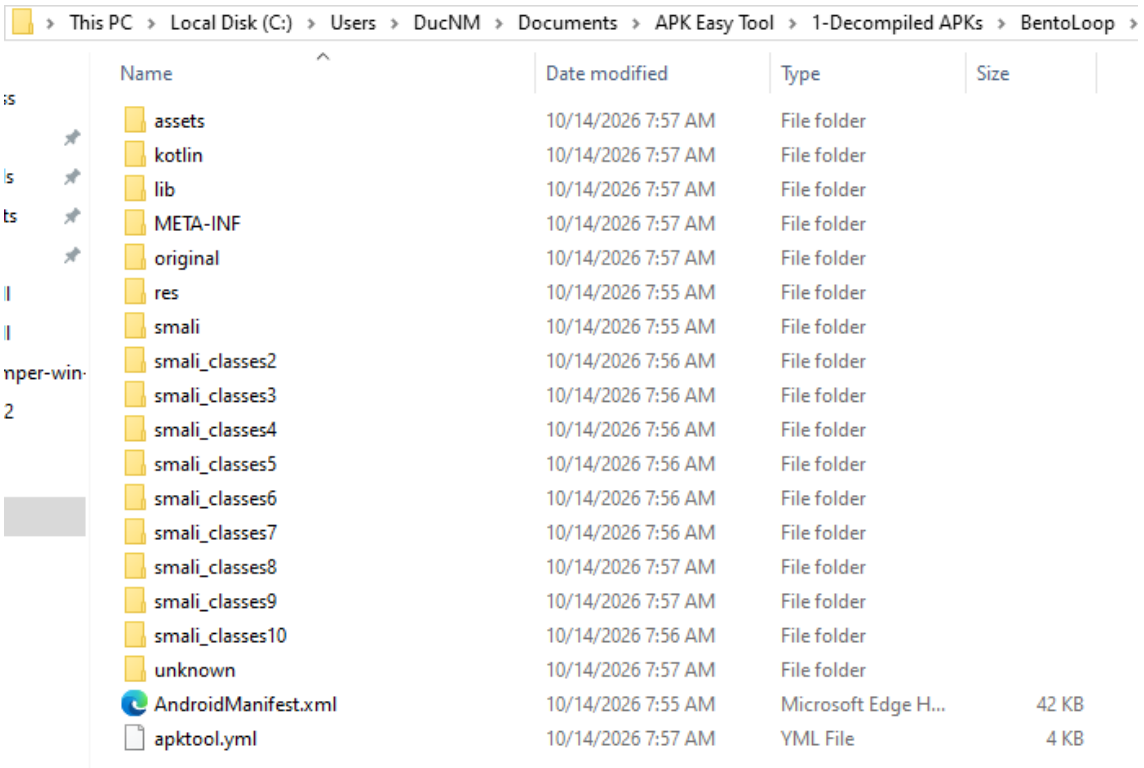
2 Decompile APK và xác định thư viện

Đầu vào là C:\Thuchanh2\BentoLoop.apk. Trong APK Easy Tool, Browse được dùng để chọn APK, sau đó bấm Decompile. Kết quả hiển thị Decompile successful. Công cụ cũng hiển thị tên package, phiên bản và thông tin SDK của ứng dụng.



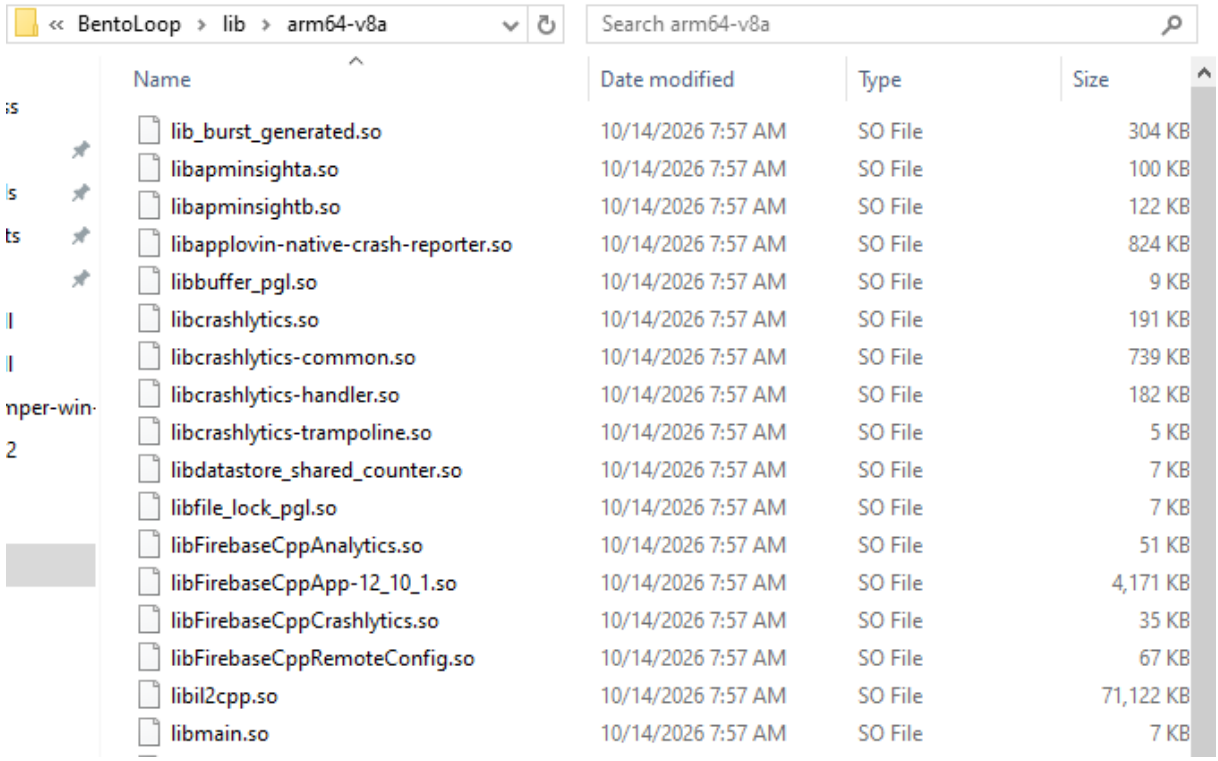
Hình 1. APK Easy Tool chọn BentoLoop.apk và báo Decompile successful

Đầu ra nằm trong C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop. Thư mục chứa assets, lib, res, các thư mục smali, AndroidManifest.xml và apktool.yml. Đây là thư mục làm việc để lấy thư viện native và dùng khi build lại APK ở nhiệm vụ tiếp theo.



Hình 2. Thư mục BentoLoop sau Decompile có lib, assets và apktool.yml

Trong lib\arm64-v8a, libil2cpp.so có dung lượng hiển thị 71 122 KB. File này được chọn để phân tích thay cho các thư viện quảng cáo, Firebase hoặc thư viện hỗ trợ khác trong cùng thư mục. Đường dẫn đầy đủ là C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\lib\arm64-v8a\libil2cpp.so.



Hình 3. libil2cpp.so trong thư mục ARM64 của BentoLoop

3 Tra cứu lớp LicenseVerifier trong dữ liệu dump

Đầu vào của bước tra cứu là gói dump có sẵn trong C:\Thuchanh2\libil2cpp_il2cppdump_20261007_0259. Các file được sử dụng gồm dump.cs,

script.json, stringliteral.json, ghidra.py và thư mục DummyDll. Thư viện dùng trong Ghidra được lấy từ APK vừa Decompile. Không thực hiện lại bước tạo dump trong phần này.

Tìm LicenseVerifier trong dump.cs cho kết quả lớp static thuộc namespace GeckoArrow.Runtime.LoadingSystem. Lớp có field private static readonly byte[] EncryptedBytes. VerifyLicense nhận string inputKey và trả về bool; CheckTransformedLicense nhận string key và cũng trả về bool. Hai hàm này là đầu mối để đọc cách kiểm tra license trong mã native.

```
// Namespace: GeckoArrow.Runtime.LoadingSystem
public static class LicenseVerifier // TypeDefIndex: 12855
{
        // Fields
        private static readonly byte[] EncryptedBytes; // 0x0

        // Methods

        // RVA: 0x1E76FA0 Offset: 0x1E72FA0 VA: 0x1E76FA0
        public static bool CheckTransformedLicense(string key) { }

        // RVA: 0x1E7689C Offset: 0x1E7289C VA: 0x1E7689C
        public static bool VerifyLicense(string inputKey) { }

        // RVA: 0x1E76EF4 Offset: 0x1E72EF4 VA: 0x1E76EF4
        public static bool IsLicenseActivated() { }

        // RVA: 0x1E769FC Offset: 0x1E729FC VA: 0x1E769FC
        public static void SaveLicenseActivated(bool activated = True) { }

        // RVA: 0x1E77110 Offset: 0x1E73110 VA: 0x1E77110
        private static void .cctor() { }
}
```

Hình 4. Khai báo LicenseVerifier, mảng EncryptedBytes và RVA của các phương thức trong dump.cs

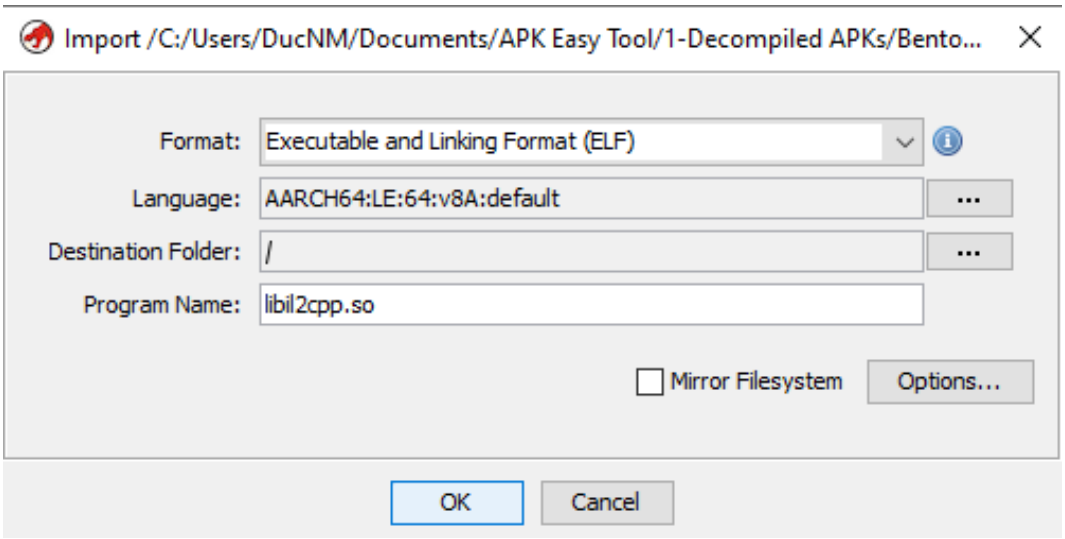
Phương thức	RVA	Địa chỉ Ghidra
VerifyLicense	0x1E7689C	0x1F7689C
CheckTransformedLicense	0x1E76FA0	0x1F76FA0
IsLicenseActivated	0x1E76EF4	0x1F76EF4
SaveLicenseActivated	0x1E769FC	0x1F769FC
.cctor	0x1E77110	0x1F77110

Các thân hàm rỗng trong dump.cs chỉ phục vụ tra cứu. Kết quả bước này là xác định đúng tên lớp, vai trò các phương thức và RVA để mở mã native. Địa chỉ Ghidra trong bảng được tính sau khi kiểm tra Image Base ở bước tiếp theo.

4 Import thư viện và gắn tên trong Ghidra

4.1 Nhận dạng thư viện ARM64

Đầu vào là libil2cpp.so trong thư mục arm64-v8a của BentoLoop. File được import vào project Thuchanh2. Ghidra nhận dạng định dạng Executable and Linking Format và language AARCH64:LE:64:v8A:default. Sau khi mở CodeBrowser, bước phân tích tự động được chạy để nhận diện lệnh, hàm và tham chiếu.



Hình 5. Hộp Import nhận dạng libil2cpp.so là ELF ARM64

Import Results Summary xác nhận processor AARCH64, little-endian, địa chỉ 64 bit và ELF shared object. MD5 của thư viện là 36834a117aa0a39f3e2a1068a5c6e846. Mã nhận diện này được dùng để phân biệt đúng file đã phân tích, không phải mã băm của toàn bộ APK.

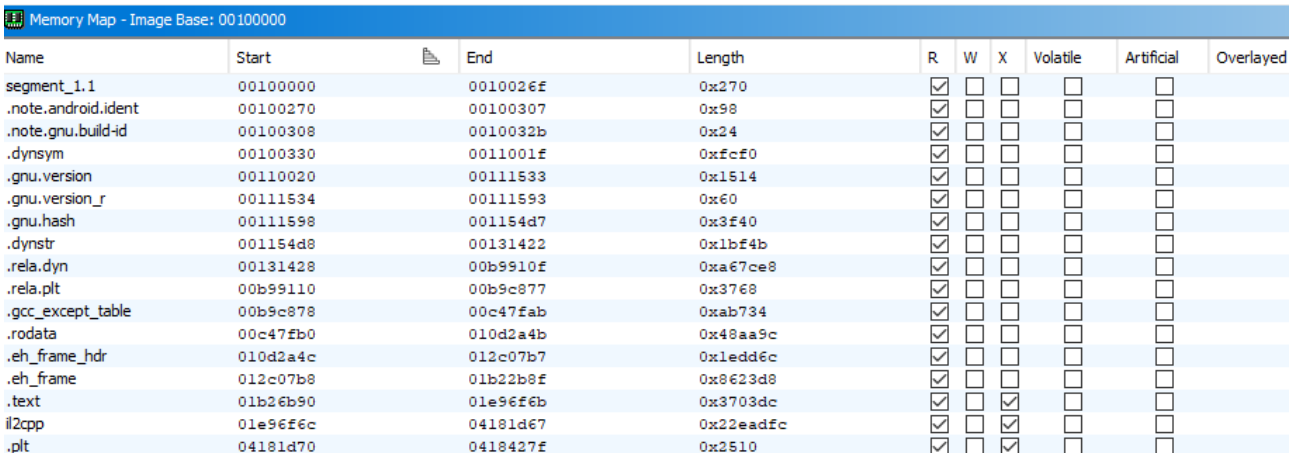
Project File Name:	libil2cpp.so
Last Modified:	Wed Oct 14 08:06:40 GMT+07:00 2026
Readonly:	false
Program Name:	libil2cpp.so
Language ID:	AARCH64:LE:64:v8A (1.6)
Compiler ID:	default
Processor:	AARCH64
Endian:	Little
Address Size:	64
Minimum Address:	00100000
Maximum Address:	_elfSectionHeaders::0000073f
# of Bytes:	75094466
# of Memory Blocks:	31
# of Instructions:	2372
# of Defined Data:	58182
# of Functions:	2142
# of Symbols:	3020
# of Data Types:	91
# of Data Type Categories:	2
Created With Ghidra Version:	12.1.2
Date Created:	Wed Oct 14 08:06:15 GMT+07:00 2026
ELF File Type:	shared object
ELF Note[GNU BuildId]:	20806e5d67715ba5d64c05300c5acfd471b8df41
ELF Original Image Base:	0x0
ELF Prelinked:	false
Elf Comment[0]:	
Elf Comment[1]:	Linker: LLD 18.0.3
Elf Comment[2]:	Android (12470979, +pgo, +bolt, +lto, +mlgo, based on r522817c) clang v
Elf Comment[3]:	Android (12470979, based on r522817c) clang version 18.0.3 (https://anc
Executable Format:	Executable and Linking Format (ELF)
Executable Location:	/C:/Users/DucNM/Documents/APK Easy Tool/1-Decompiled APKs/BentoLoop/lib
Executable MD5:	36834a117aa0a39f3e2a1068a5c6e846
Executable SHA256:	9fe4f27d2e1ec694bfff6ec6b9dbf98e60f51fb021c24ebae30b7dc8b18308a3
FSRL:	file:///C:/Users/DucNM/Documents/APK Easy Tool/1-Decompiled APKs/BentoI
GNU DebugLink:	libil2cpp.sym.so

Hình 6. Thông tin kiến trúc và mã nhận diện thư viện trong Import Results Summary

4.2 Quy đổi RVA sang địa chỉ Ghidra

Memory Map hiển thị Image Base 00100000, tương ứng 0x100000. Vì vậy, RVA trong dump cần cộng thêm Image Base để mở đúng vị trí trong chương trình đã import. Với VerifyLicense, kết quả là 0x1F7689C. File Offset là vị trí trong file, không dùng thay cho địa chỉ Ghidra.

```
Địa chỉ Ghidra = Image Base + RVA
0x100000 + 0x1E7689C = 0x1F7689C
```

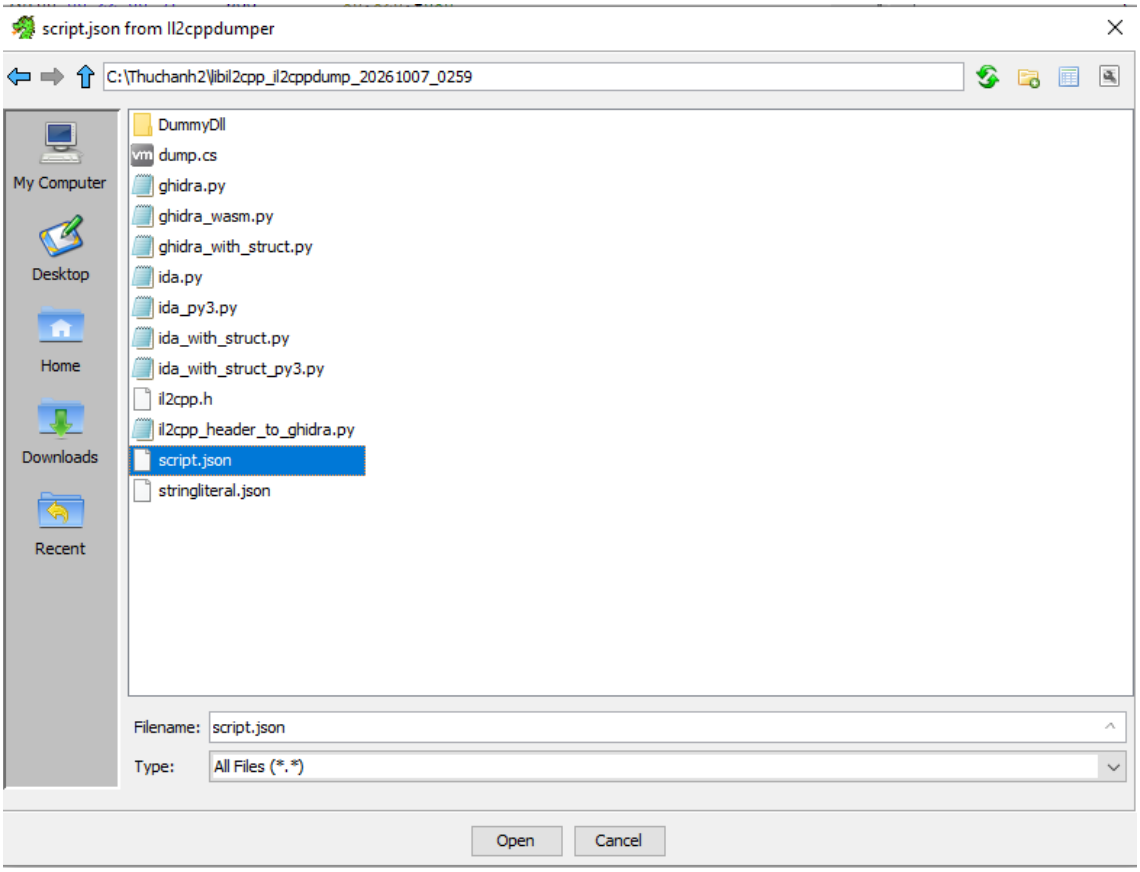


Name	Start	End	Length	R	W	X	Volatile	Artificial	Overlayed
segment_1.1	00100000	0010026f	0x270	☑	☐	☐	☐	☐	
.note.android.ident	00100270	00100307	0x98	☑	☐	☐	☐	☐	
.note.gnu.build-id	00100308	0010032b	0x24	☑	☐	☐	☐	☐	
.dynsym	00100330	0011001f	0xfcf0	☑	☐	☐	☐	☐	
.gnu.version	00110020	00111533	0x1514	☑	☐	☐	☐	☐	
.gnu.version_r	00111534	00111593	0x60	☑	☐	☐	☐	☐	
.gnu.hash	00111598	001154d7	0x3f40	☑	☐	☐	☐	☐	
.dynstr	001154d8	00131422	0x1bf4b	☑	☐	☐	☐	☐	
.rela.dyn	00131428	00b9910f	0xa67ce8	☑	☐	☐	☐	☐	
.rela.plt	00b99110	00b9c877	0x3768	☑	☐	☐	☐	☐	
.gcc_except_table	00b9c878	00c47fab	0xab734	☑	☐	☐	☐	☐	
.rodata	00c47fb0	010d2a4b	0x48aa9c	☑	☐	☐	☐	☐	
.eh_frame_hdr	010d2a4c	012c07b7	0x1edd6c	☑	☐	☐	☐	☐	
.eh_frame	012c07b8	01b22b8f	0x8623d8	☑	☐	☐	☐	☐	
.text	01b26b90	01e96f6b	0x3703dc	☑	☐	☑	☐	☐	
il2cpp	01e96f6c	04181d67	0x22eadfc	☑	☐	☑	☐	☐	
.plt	04181d70	0418427f	0x2510	☑	☐	☑	☐	☐	

Hình 7. Memory Map xác nhận Image Base 0x100000 và các vùng bộ nhớ

4.3 Áp dụng tên từ script.json

Thư mục chứa ghidra.py được thêm vào Script Manager. Khi chạy script, script.json trong gói dump Bento được chọn làm đầu vào. Dữ liệu này gắn tên các phương thức và chuỗi vào địa chỉ tương ứng. Đầu ra thể hiện ở Decompiler với tên đầy đủ như GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$VerifyLicense và các nhãn StringLiteral.



Hình 8. Chọn script.json của Bento làm đầu vào cho script gắn tên

Trước khi gắn tên, Ghidra dùng tên tự động FUN_01f7689c. Sau khi gắn tên, cùng địa chỉ đó hiển thị VerifyLicense. Việc thay đổi nhãn làm mã giả dễ đọc hơn, không thay đổi mã máy trong thư viện. File gamebento.py được cung cấp cũng là script nhập dữ liệu script.json; chức năng của nó là gắn nhãn, chú thích và tạo hàm, không giải mã key.

5 Phân tích luồng kiểm tra license

5.1 Kiểm tra đầu vào và nhánh so sánh trực tiếp

Đầu vào của bước phân tích là hàm VerifyLicense tại 0x1F7689C. Hàm được mở bằng Go To, sau đó đọc trong Decompiler. Phần đầu gọi String.IsNullOrWhiteSpace để loại đầu vào rỗng hoặc chỉ có khoảng trắng. Với đầu vào hợp lệ, String.Trim tạo chuỗi đã bỏ khoảng trắng ở hai đầu.

Tiếp theo, String.Equals so sánh chuỗi đã Trim với StringLiteral_2770, truyền giá trị 4 ở tham số kiểu so sánh. Khi kết quả khác 0, luồng đi đến nhánh ngoài phần kiểm tra bổ sung. Khi kết quả bằng 0, VerifyLicense gọi CheckTransformedLicense, rồi có thể gọi SecurityAuthService.DispatchVerification nếu nhánh trước chưa chấp nhận đầu vào.

```
  }
  uVar2 = System.String$$IsNullOrWhiteSpace(param_1,0);
  if ((uVar2 & 1) == 0) {
    if (param_1 == 0) {
                    /* WARNING: Subroutine does not return */
      FUN_01da5340();
    }
    uVar3 = System.String$$Trim(param_1,0);
    uVar2 = System.String$$Equals(uVar3,StringLiteral_2770,4,0);
    if ((uVar2 & 1) == 0) {
      if (*(int *)(GeckoArrow.Runtime.LoadingSystem.LicenseVerifier_TypeInfo + 0xe4) == 0) {
        thunk_FUN_01e01e10();
      }
      uVar2 = GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$CheckTransformedLicense(uVar3);
      if ((uVar2 & 1) == 0) {
        uVar2 = GeckoArrow.Runtime.LoadingSystem.SecurityAuthService$$DispatchVerification(uVar3);
        if ((uVar2 & 1) == 0) goto LAB_01f76914;
        iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
```

Hình 9. VerifyLicense chuẩn hóa đầu vào rồi so sánh với StringLiteral_2770

5.2 Nhánh trả về thành công

Trong phần cuối hàm, nhánh chuỗi khớp với StringLiteral_2770 chọn StringLiteral_1120 để ghi log. Luồng sau đó gán giá trị trả về bằng 1. Đầu vào không được chấp nhận bởi các nhánh kiểm tra chuyển đến LAB_01f76914 và trả về 0. Như vậy, có thể tìm key ở chuỗi được so sánh trực tiếp trước khi giải nhánh mã hóa.

```
        iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
        puVar4 = &StringLiteral_11021;
      }
    }
    else {
      iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
      puVar4 = &StringLiteral_11020;
    }
    if (iVar1 == 0) {
      thunk_FUN_01e01e10();
    }
    UnityEngine.Debug$$Log(*puVar4,0);
    uVar3 = 1;
  }
  else {
LAB_01f76914:
    uVar3 = 0;
  }
  return uVar3;
}
```

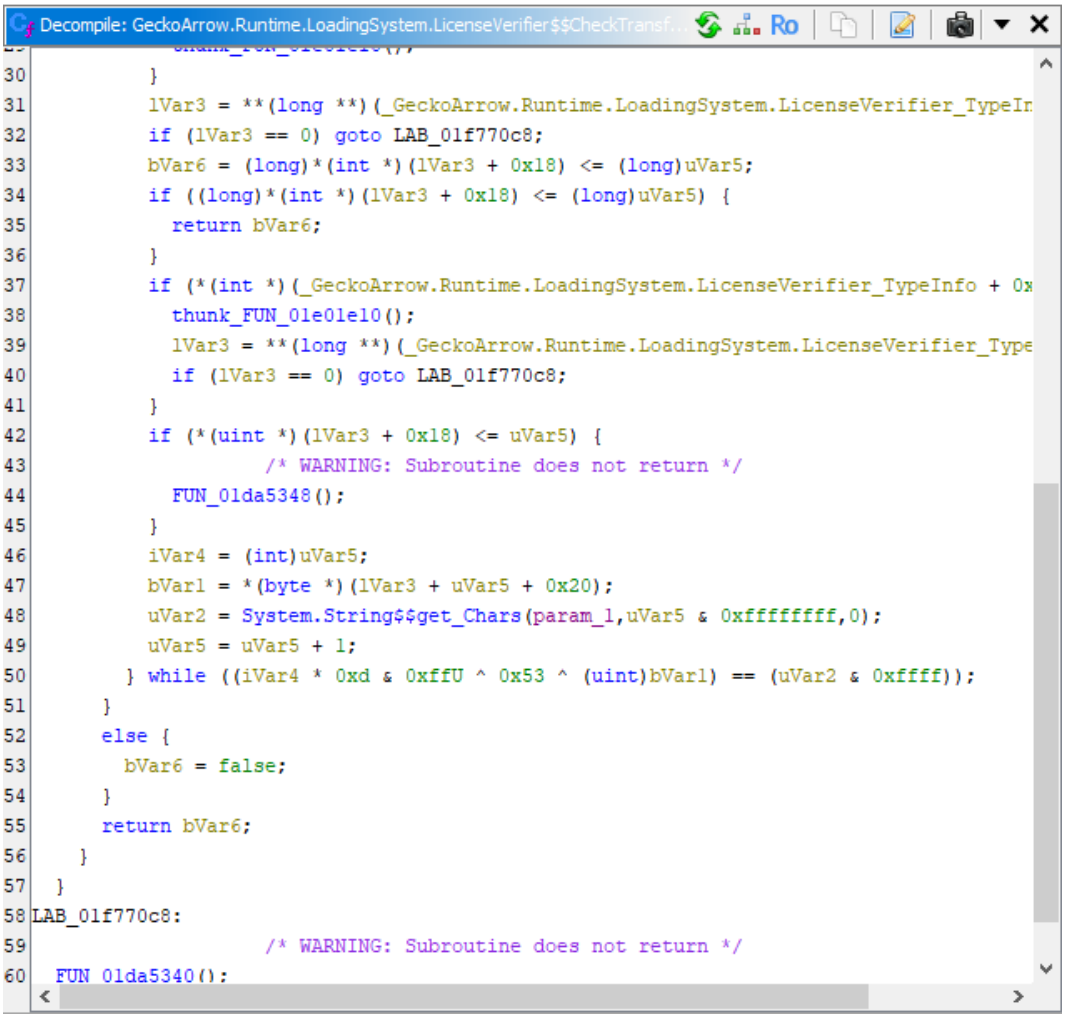
Hình 10. Nhánh so sánh khớp dẫn đến giá trị trả về 1 trong VerifyLicense

Luồng chính có thể viết lại bằng mã giả dưới đây. Đoạn này mô tả điều kiện đã quan sát, không phải mã nguồn C# gốc.

```
if (IsNullOrWhiteSpace(inputKey)) return false;
key = Trim(inputKey);
if (Equals(key, StringLiteral_2770, 4)) return true;
if (CheckTransformedLicense(key)) return true;
return SecurityAuthService.DispatchVerification(key);
```

5.3 Nhánh kiểm tra mảng mã hóa

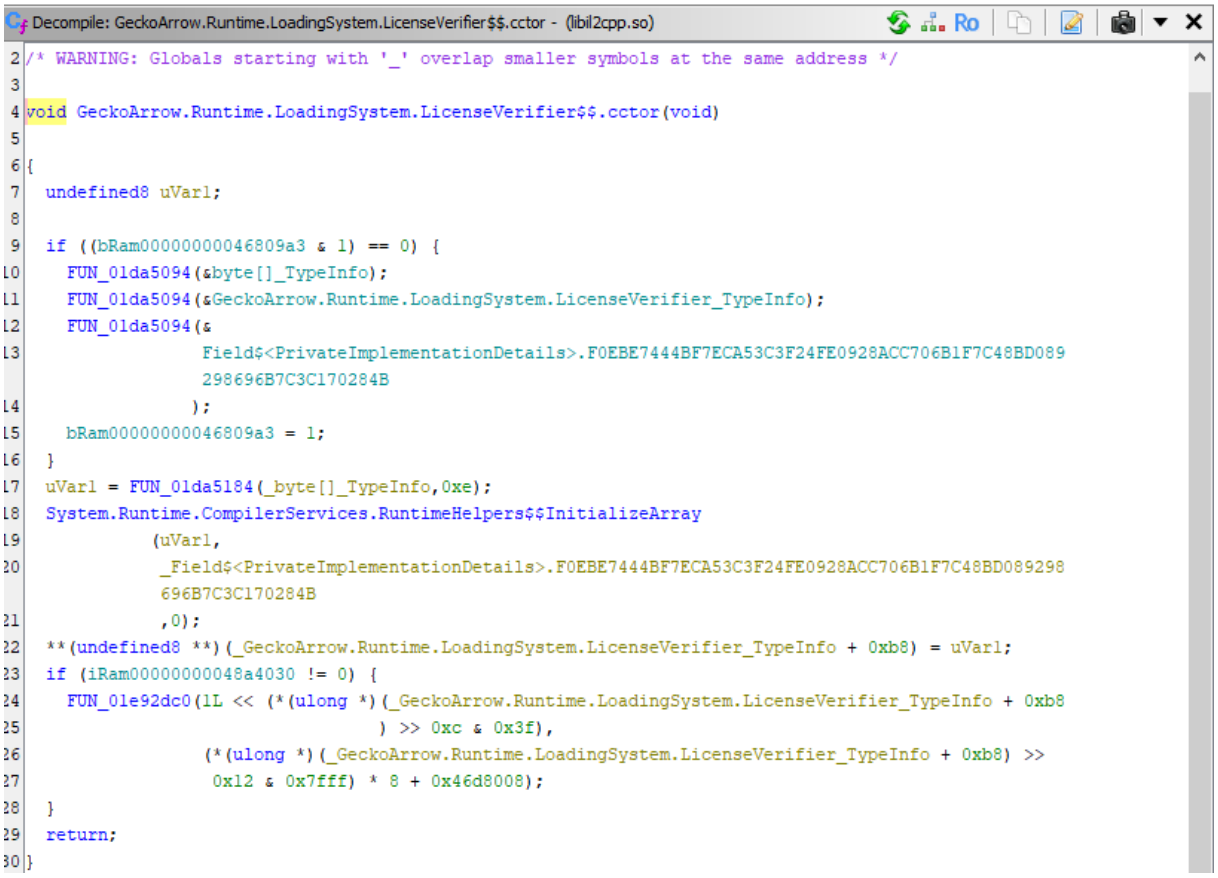
CheckTransformedLicense tại 0x1F76FA0 kiểm tra độ dài chuỗi bằng độ dài mảng dữ liệu, sau đó duyệt từng vị trí. Vòng lặp lấy một byte trong mảng và một ký tự bằng String.get_Chars. Phép đối chiếu sử dụng XOR với 0x53 và giá trị (i × 13) AND 0xFF, với i bắt đầu từ 0.



Hình 11. Vòng lặp trong CheckTransformedLicense đối chiếu ký tự bằng phép XOR

```
ký_tự[i] = EncryptedBytes[i] XOR 0x53 XOR ((i × 13) AND 0xFF)
```

Hàm khởi tạo lớp .cctor tại 0x1F77110 tạo mảng byte dài 0x1E, tức 30 byte, rồi gọi RuntimeHelpers.InitializeArray để nạp dữ liệu từ field của PrivateImplementationDetails. Các byte tại nhãn tham chiếu field chưa phải nội dung mảng. Trong bài này, nhánh so sánh trực tiếp đã cung cấp key được game chấp nhận nên chưa thực hiện giải toàn bộ mảng 30 byte.

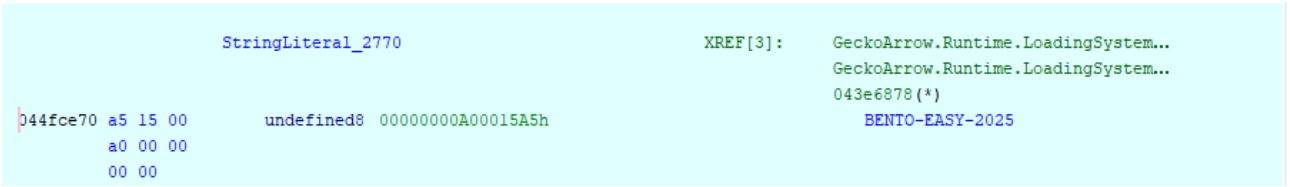


Hình 12. Hàm .cctor tạo mảng 30 byte và gọi RuntimeHelpers.InitializeArray

6 Đối chiếu key và kiểm chứng trên APK gốc

6.1 Đọc nội dung StringLiteral_2770

Đầu vào là nhãn StringLiteral_2770 trong lời gọi String.Equals của VerifyLicense. Bấm đúp nhãn trong Decompiler chuyển Listing đến 044fce70. Chú thích tại đây hiển thị BENTO-EASY-2025. Đây là chuỗi được dùng ở nhánh so sánh trực tiếp, không phải tên field EncryptedBytes hoặc khóa lưu trạng thái BENTO_LICENSE_ACTIVATED ở vị trí khác.



Hình 13. StringLiteral_2770 tại 0x044FCE70 có nội dung BENTO-EASY-2025

6.2 Đối chiếu với stringliteral.json

Trong stringliteral.json của cùng gói dump, tìm 43FCE70 cho mục có value là BENTO-EASY-2025 và address là 0x43FCE70. Cộng Image Base 0x100000 cho địa chỉ này được 0x44FCE70, khớp Listing. Cách hiển thị 044fce70 của Ghidra có thêm số 0 đầu và không ghi tiền tố 0x; đây vẫn là cùng một địa chỉ.

```
0x043FCE70 + 0x00100000 = 0x044FCE70
```

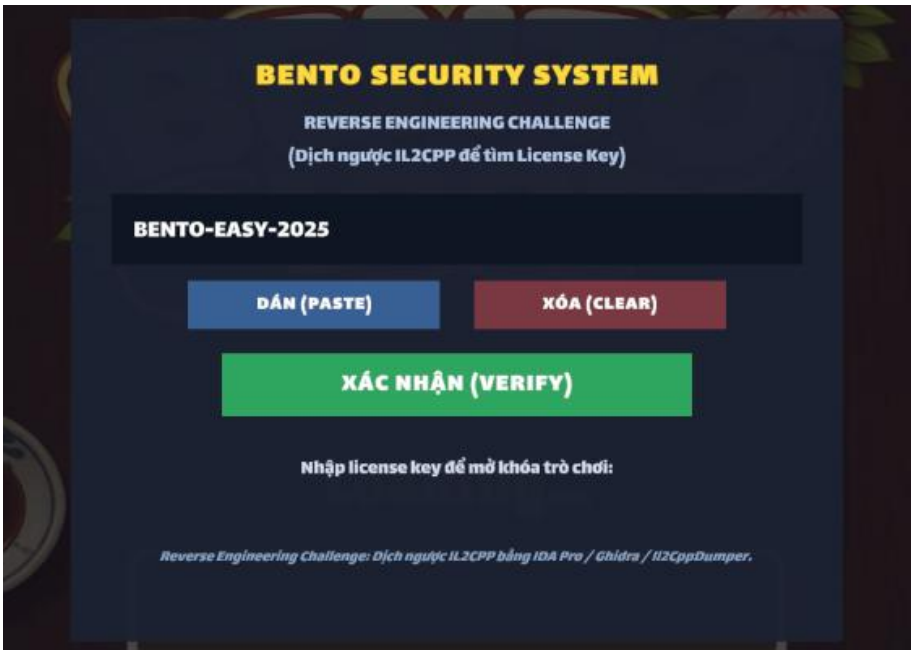
```
{
  "value": "BENTO-EASY-2025",
  "address": "0x43FCE70"
},
```

Hình 14. Mục value và address của key trong stringliteral.json

Chú thích Ghidra được script tạo từ dữ liệu dump nên việc mở JSON là đối chiếu nguồn của nhãn. Bằng chứng về vai trò của chuỗi nằm ở nhánh String.Equals và giá trị trả về trong VerifyLicense; kết quả chạy trên APK gốc xác nhận key có hiệu lực.

6.3 Nhập key và vượt màn hình license

Đầu vào thử nghiệm là APK gốc chưa sửa và key BENTO-EASY-2025 đã xác định. Chuỗi được nhập vào ô license trên màn hình BENTO SECURITY SYSTEM, sau đó bấm XÁC NHẬN (VERIFY). Ảnh trước khi xác nhận thể hiện đúng key trong ô nhập.



Hình 15. Nhập BENTO-EASY-2025 vào màn hình kiểm tra license của APK gốc

Sau khi xác nhận, màn hình license được vượt qua và game hiển thị Level 1. Kết quả này hoàn tất nhiệm vụ tìm một key được chấp nhận và kiểm chứng trên APK gốc. Kết quả chưa chứng minh rằng các chức năng khác đã được mở khóa hoặc mọi chuỗi nhập đều được chấp nhận.



Hình 16. Bento Loop vào Level 1 sau khi xác nhận license

7 Trạng thái nhiệm vụ sửa và build APK

Nhiệm vụ tìm key đã hoàn thành bằng nhánh so sánh trực tiếp và được kiểm chứng trên APK gốc. Chưa thực hiện patch hàm, thay libil2cpp.so, Compile, ZipAlign hoặc ký một APK đã sửa. Vì vậy, kết quả hiện tại chưa đáp ứng nhiệm vụ thứ hai là vòng tránh cơ chế kiểm tra và build lại APK.

Theo hướng dẫn công cụ, bước build tiếp theo sử dụng thư mục BentoLoop đã Decompile và có apktool.yml. Sau khi có thư viện đã sửa, libil2cpp.so trong lib\arm64-v8a được thay bằng bản đó, rồi chọn Select decompiled APK và Compile. APK kết quả cần được căn chỉnh và ký nếu công cụ chưa thực hiện tự động. Đây là quy trình cần thực hiện tiếp, chưa phải kết quả đã đạt.

8 Nhận xét an toàn và phương án phòng tránh

8.1 Rủi ro của nhánh so sánh key trên client

BENTO-EASY-2025 được lưu dưới dạng chuỗi và được so sánh ngay trong thư viện phía client. Sau khi có dữ liệu dump và tên phương thức, có thể lần từ VerifyLicense đến chuỗi mà không cần giải nhánh EncryptedBytes. Việc đặt thêm một nhánh XOR không bảo vệ được key ở nhánh so sánh trực tiếp vẫn còn tồn tại.

Một khả năng can thiệp khác là thay đổi kết quả trả về hoặc nhánh điều kiện trong hàm kiểm tra. Khả năng này chưa được thử nghiệm bằng APK đã patch trong bài hiện tại. Chưa đủ dữ liệu để kết luận game có cơ chế kiểm tra tính toàn vẹn hoặc xác thực bổ sung ngoài luồng đã xem hay không.

8.2 Phương án giảm rủi ro

Với ứng dụng có máy chủ, license nên được xác minh trên máy chủ theo tài khoản hoặc quyền sử dụng. Client nhận kết quả hoặc token có thời hạn thay vì giữ một key chung để so sánh. Không đặt key bí mật hoặc nhánh chấp nhận thử nghiệm trong bản phát hành; các thao tác cần quyền sử dụng cũng phải được kiểm tra, thay vì chỉ khóa ở màn hình khởi động.

Nếu cần hoạt động offline, có thể dùng dữ liệu license được ký số. Client giữ khóa công khai để kiểm tra chữ ký, còn khóa riêng dùng để phát hành license được giữ ngoài ứng dụng. Cách này tránh nhúng một bí mật chung để người dùng có thể đọc ra, nhưng mã kiểm tra phía client vẫn có thể bị patch nên cần kết hợp kiểm tra tính toàn vẹn và giới hạn phù hợp với môi trường offline.

9 Kết luận

Qua bài thực hành, đã sử dụng APK Easy Tool để lấy thư viện native, đọc dump.cs để xác định lớp và RVA, rồi dùng Ghidra để đối chiếu địa chỉ và lần theo nhánh kiểm tra license. Kết hợp mã giả với stringliteral.json đã xác định được BENTO-EASY-2025. Thử nghiệm nhập key trên APK gốc cho kết quả vượt màn hình license và vào Level 1.

Các bước đã thực hiện cho thấy có thể dùng bộ công cụ này để tìm điều kiện chấp nhận đầu vào, phân biệt tên nhãn với nội dung dữ liệu và kiểm chứng kết quả bằng việc chạy ứng dụng. Phần sửa thư viện để vòng tránh kiểm tra và build lại APK vẫn là nhiệm vụ cần thực hiện tiếp.